Chapitre 7 : Récursivité

ITC - 2025/2026

Félix POTIÉ

Sommaire

Sommaire 2
Exemple : factorielle 4
Principe de pile d'execution 7
Exemple : une suite mathématique 17
Exemple : suite de Fibonacci 20
Lien avec diviser pour régner 24

Exemples

Exemple : factorielle

La factorielle d'un entier naturel n, notée n! est définie par :

- 0! = 1
- $\forall n \in \mathbb{N}^*, n! = n * (n-1)!$

On peut définir une fonction itérative (classique) suivante :

```
def factorielle_iterative(n):
    res = 1
    for i in range(2, n + 1):
        res *= i
    return res
```

Mais on peut aussi définir une fonction récursive :

Exemple : factorielle - 2

Définition 1: Fonction récursive.

Une fonction est dite récursive si elle s'appelle elle-même dans son propre corps.
La fonction, après avoir été mise en mémoire, est capable de s'appeler autant de fois que nécessaire (jusqu'à une certaine limite) afin d'arriver au résultat direct qu'elle connaît, puis de remonter au résultat attendu.

```
def facto(n):
    if n == 0:
        return 1
    else:
        return n * facto(n-1)
```

Exemple : factorielle - 3

Le principe d'exécution est le suivant, par exemple pour 4! :

- `facto(4) : 4 != 0`, je calcule `4 * facto(3)`
  - `facto(3) : 3 != 0`, je calcule `3 * facto(2)`
    - `facto(2) : 2 != 0`, je calcule `2 * facto(1)`
      - `facto(1) : 1 != 0`, je calcule `1 * facto(0)`
        - `facto(0) : 0 == 0`, c'est 1. Je renvoie 1.
      - J'ai reçu 1, donc `1 * 0! = 1 * 1 = 1`. Je renvoie 1.
    - J'ai reçu 1, donc `2 * 1! = 2 * 1 = 2`. Je renvoie 2.
  - J'ai reçu 2, donc `3 * 2! = 3 * 2 = 6`. Je renvoie 6.
- J'ai reçu 6, donc `4 * 3! = 4 * 6 = 24`. Je renvoie 24.

Cette remontée des appels est possible grâce à la pile d'execution.

Principe de pile d’execution

Définition 2: Pile.

Une pile en informatique est une structure de donnée où l’on peut :

- Ajouter un élément au sommet de la pile
- Récupérer l’élément au sommet de la pile

On peut voir ça comme une pile d’assiettes.

Définition 3: Pile d'exécutions.

On appelle pile d’exécutions (ou pile d’appels), une pile qui va être initialiser à l“exécution d’un programme. Cette pile à pour objectif de garder en mémoire le chemin d’exécution des fonctions de notre programme.

Principe de pile d'execution - 2

```
def h(n):
    return n - 5

def g(n):
    x = 2 * n
    return h(x)

def f(n):
    x = g(n)
    return h(x)

print(f(2))
```

Principe de pile d'execution - 3

Lorqu'on a une erreur on voit le chemin (la pile) de l'exécution.

```python
def h(n):
    return 1/n

def g(n):
    x = 2 * n
    return h(x)

def f(n):
    x = g(n)
    return h(x)

print(f(0))
```

```
Traceback (most recent call last):
  File "<basthon-input-3-9bea131f8680>", line 12, in <module>
    print(f(0))
          ^^^^
  File "<basthon-input-3-9bea131f8680>", line 9, in f
    x = g(n)
        ^^^^
  File "<basthon-input-3-9bea131f8680>", line 6, in g
    return h(x)
           ^^^^
  File "<basthon-input-3-9bea131f8680>", line 2, in h
    return 1/n
           ~^~
ZeroDivisionError: division by zero
```

Principe de pile d'execution - 4

Pareil pour une fonction récursive, chaque appel de la fonction est empilé dans la pile d'exécution.

```python
def facto(n):
    if n==0:
        return 1/0 # on créer volontairement une erreur
    else:
        return n * facto(n-1)
facto(5)
```

```
Traceback (most recent call last):
  File "<basthon-input-2-25ce2053f554>", line 6, in <module>
    facto(5)
  File "<basthon-input-2-25ce2053f554>", line 5, in facto
    return n * facto(n-1)
               ^^^^^^^^^^
  File "<basthon-input-2-25ce2053f554>", line 5, in facto
    return n * facto(n-1)
               ^^^^^^^^^^
  File "<basthon-input-2-25ce2053f554>", line 5, in facto
    return n * facto(n-1)
               ^^^^^^^^^^
  [Previous line repeated 2 more times]
  File "<basthon-input-2-25ce2053f554>", line 3, in facto
    return 1/0 # on créer volontairement une erreur
           ~^~
ZeroDivisionError: division by zero
```

Principe de pile d'execution - 5

Exercice : Listez dans l'ordre les appels de fonctions survenu pour l'exécution du programme suivant, ainsi que le résultat affiché.

```
def f(x):
    y = x + 1
    return x * y

def g(x, y):
    z = f(x)
    return z + f(y)

def h(x, y):
    return f(x) + g(x, y)

print(h(1,2))
```

Principe de pile d'execution - 6

```
h(1,2) :
    f(1) + g(1,2) :
        f(1) -> 2
        g(1,2) :
            f(1) + f(2) -> 2 + 6
        -> 8
    -> 8 + 2
-> 10
```

Principe de pile d'execution - 7

Exercice : De la même façon, listez les appels de fonctions survenu ainsi que le résultat du programme suivant :

```
def rec(n, k):
    if n == 0 :
        return k
    return k + rec(n-1, k*2)

print(rec(3, 2))
```

Principe de pile d'execution - 8

```
rec(3, 2) :
    2 + rec(2, 4) :
        4 + rec(1, 8) :
            8 + rec(0, 16) :
            -> 8 + 16 = 24
        -> 4 + 24 = 28
    -> 2 + 28 = 30
-> 30
```

Principe de pile d'execution - 9

Attention, comme pour une boucle while, une fonction récursive peut ne jamais terminer (en principe).

```
def sans_fin(n):
    if n==1:
        return 0
    else:
        return sans_fin(n/10)
```

On voit bien que cet algorithme ne peut fonctionner que si `n` est une puissance de 10. L'exécution de cet algorithme pour un nombre qui n'est pas puissance de 10 va conduire à une boucle infinie.

Principe de pile d'execution - 10

La pile d'appel n'a pas une mémoire infinie. En python elle est limitée par défaut à environ 1000 appels imbriqués.

```python
def f(n):
    if n == 0:
        return 0
    return f(n-1)

f(1000)
```

```
Traceback (most recent call last):
  File "<basthon-input-15-e82b639aee6e>", line 6, in <module>
    f(1000)
  File "<basthon-input-15-e82b639aee6e>", line 4, in f
    return f(n-1)
           ^^^^^^
  File "<basthon-input-15-e82b639aee6e>", line 4, in f
    return f(n-1)
           ^^^^^^
  File "<basthon-input-15-e82b639aee6e>", line 4, in f
    return f(n-1)
           ^^^^^^
  [Previous line repeated 990 more times]
RecursionError: maximum recursion depth exceeded
```

Exemple : une suite mathématique

Soit la suite (u_n) suivante, qui calcule une approximation de $\sqrt{3}$:

- $u_0 = 2$
- $\forall n \in \mathbb{N}^*, u_n = \frac{1}{2}\left(u_{n-1} + \frac{3}{u_{n-1}}\right)$

Exercice : Écrire une version récursive de cette suite puis une version itérative. Laquelle est la plus naturelle à écrire ?

Exemple : une suite mathématique - 2

Il est plus naturel d'écrire une fonction récursive pour cette suite :

```
def u(n):
    if n == 0:
        return 2
    else:
        x = u(n-1)
        return 0.5 * (x + 3/x)
```

Exemple : une suite mathématique - 3

Sans récursivité, on peut écrire :

```
def u_iteratif(n):
    res = 2
    for i in range(1, n+1):
        res = 0.5 * (res + 3 / res)
    return res
```

Exemple : suite de Fibonacci

La suite de Fibonacci (F_n) est définie par :

- $F_0 = 0$
- $F_1 = 1$
- $\forall n \in \mathbb{N}^*, F_n = F_{n-1} + F_{n-2}$

Exercice : Écrire une version récursive de cette fonction.

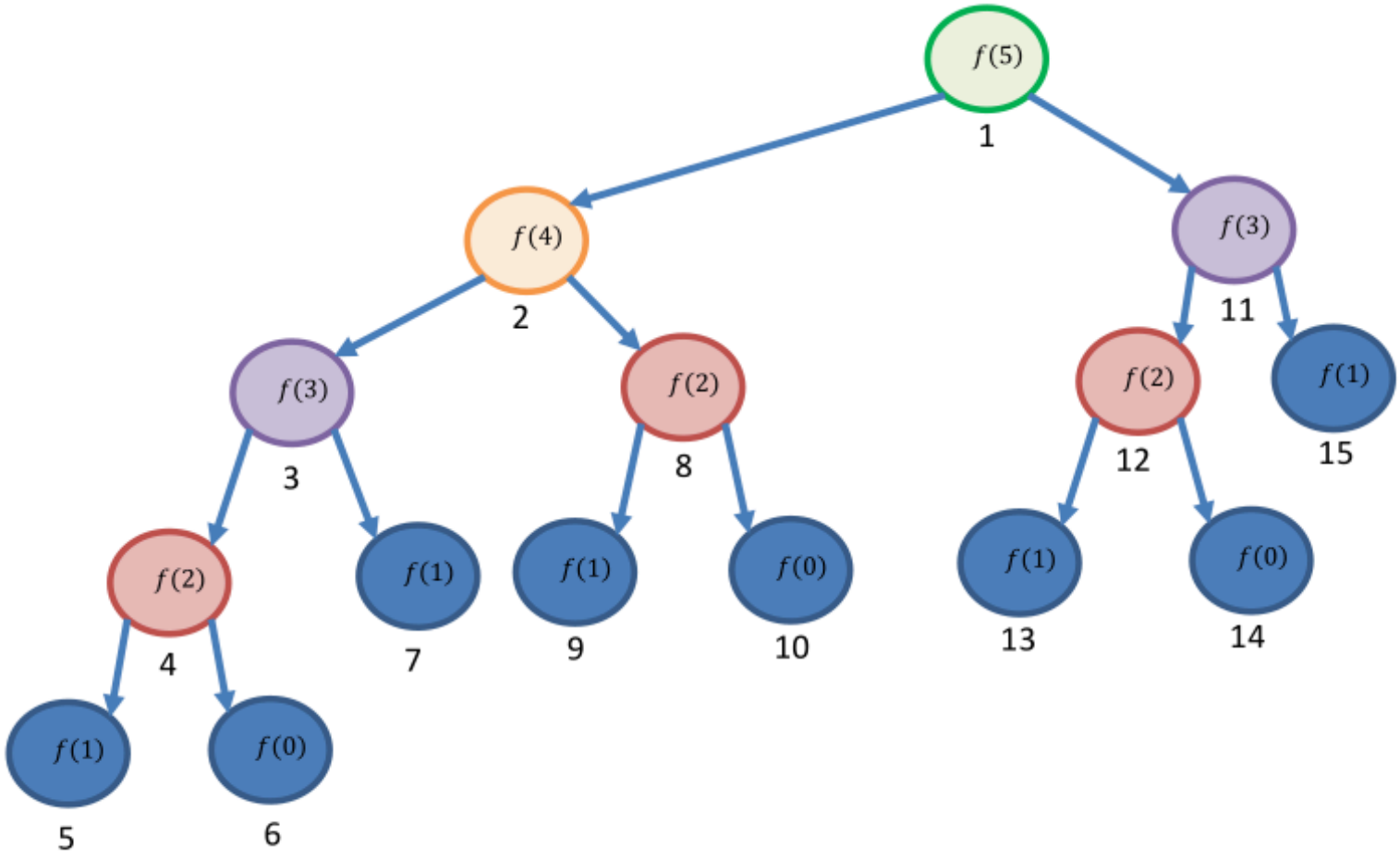
Exemple : suite de Fibonacci - 2

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)
```

Attention cependant, cette version est très inefficace car elle recalcule plusieurs fois les mêmes valeurs !

Par exemple, pour calculer `fibonacci(5)`, on calcule `fibonacci(3)` deux fois (une fois pour `fibonacci(4)` et une autre fois pour `fibonacci(2)`)...

Exemple : suite de Fibonacci - 3



Complexité : $O(2^n)$ (on verra comment la calculer plus tard)

Exemple : suite de Fibonacci - 4

Parfois il est préférable d'utiliser une approche itérative ou de la programmation dynamique (que vous verrez l'an prochain) pour éviter ces recalculs inutiles.

```
def fibonacci_iteratif(n):
    if n in [0,1]:
        return n
    f = [0,1]
    for i in range(2, n + 1):
        f.append(f[i-1] + f[i-2])
    return f[n]
```

Complexité ?

Lien avec diviser pour régner

La récursivité est la base de la stratégie diviser pour régner

Définition 4: Méthode diviser pour régner.

- Traiter les cas de base
- Diviser : le problème à traiter est divisé en sous problèmes plus simples
- Régner : on applique récursivement l'algorithme à chaque sous problème
- Combiner : on trouve la solution du problème initial en combinant les différents résultatsintermédiaires

Exercice : Implémenter la recherche dichotomique dans une liste trié avec une fonciton récursive. Faites une premières version avec des slices qui renvoie juste True ou False.

Lien avec diviser pour régner - 2

```
def rech_dicho_slice(liste, x):
    if len(liste) == 0:
        return False  # x non trouvée
    milieu = len(liste) // 2
    if liste[milieu] == x:
        return True
    elif liste[milieu] < x:
        return rech_dicho_slice(liste[milieu+1:], x)
    else:
        return rech_dicho_slice(liste[:milieu], x)
```

Exercice : Maintenant, faites une version plus efficace qui utilise des indices de début et de fin. Sa signature sera donc : `rech_dicho(liste, x, debut, fin)`

Lien avec diviser pour régner - 3

```
def dicho_aux(liste, x, debut, fin):
    if debut > fin :
        return False
    milieu = (debut + fin) // 2
    if liste[milieu] == x :
        return True
    elif liste[milieu] < x :
        return dicho_aux(liste, x, debut, milieu-1)
    else :
        return dicho_aux(liste, x, milieu+1, fin)

def rech_dicho(liste, x):
    return dicho_aux(liste, x, 0, len(liste)-1)
```